KNOWLEDGE GRAPH PIPELINE

From web crawling to RAG over SPARQL

Final project report

Emilie POUPAT – Justine SCHUTTERLE - DIA 5

1. Data Acquisition & Information Extraction

1.1 Domain Definition and Seed URLs

The chosen domain for this project is the Olympics. This domain was selected for its rich relational structure, public availability of structured and unstructured data, and the presence of existing reference ontologies that can serve as alignment targets. Indeed, we can easily link entities such as cities, athletes, medals, summer, and winter. Because sport is an essential and globally relevant subject, it serves as a pertinent use case for semantic web mining.

The crawl was seeded from the following entry points defined in the crawl.py configuration, categorised by sub-domains:

- Summer: General pages and specific editions (e.g., Summer Olympic Games, 2024 Summer Olympics, Swimming at the Summer Olympics).
- Winter: Winter-specific events and editions (e.g., Winter Olympic Games, 2022 Winter Olympics, Alpine skiing).
- Athletes: Prominent historical and contemporary figures (e.g., Usain Bolt, Simone Biles, Michael Phelps).
- Host Cities: Locations that have hosted or will host the games (e.g., Paris, Los Angeles, Tokyo).

We can find it in the *./src/crawl* folder at the beginning:

```
DOMAIN_SEEDS = {
    "summer": [
        "https://en.wikipedia.org/wiki/Summer_Olympic_Games",
        "https://en.wikipedia.org/wiki/2024_Summer_Olympics",
        "https://en.wikipedia.org/wiki/2020_Summer_Olympics",
        "https://en.wikipedia.org/wiki/2016_Summer_Olympics",
        "https://en.wikipedia.org/wiki/International_Olympic_Committee",
        "https://en.wikipedia.org/wiki/Olympic_symbols",
        "https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics",
        "https://en.wikipedia.org/wiki/Swimming_at_the_Summer_Olympics",
        "https://en.wikipedia.org/wiki/Gymnastics_at_the_Summer_Olympics",
        "https://en.wikipedia.org/wiki/Olympic_Games",
    ],
    "winter": [
        "https://en.wikipedia.org/wiki/Winter_Olympic_Games",
        "https://en.wikipedia.org/wiki/2022_Winter_Olympics",
        "https://en.wikipedia.org/wiki/2018_Winter_Olympics",
        "https://en.wikipedia.org/wiki/Alpine_skiing_at_the_Winter_Olympics",
        "https://en.wikipedia.org/wiki/Figure_skating_at_the_Winter_Olympics",
        "https://en.wikipedia.org/wiki/Biathlon_at_the_Winter_Olympics",
        "https://en.wikipedia.org/wiki/Ice_hockey_at_the_Winter_Olympics",
        "https://en.wikipedia.org/wiki/Cross-country_skiing_at_the_Winter_Olympics",
    ],
    "athletes": [
        "https://en.wikipedia.org/wiki/Usain_Bolt",
        "https://en.wikipedia.org/wiki/Michael_Phelps",
        "https://en.wikipedia.org/wiki/Simone_Biles",
        "https://en.wikipedia.org/wiki/Eliud_Kipchoge",
        "https://en.wikipedia.org/wiki/Cathy_Freeman",
        "https://en.wikipedia.org/wiki/Carl_Lewis",
        "https://en.wikipedia.org/wiki/Nadia_Comaneci",
        "https://en.wikipedia.org/wiki/Jesse_Owens",
        "https://en.wikipedia.org/wiki/Yelena_Isinbayeva",
        "https://en.wikipedia.org/wiki/Mikaela_Shiffrin",
    ],
    "host_cities": [
        "https://en.wikipedia.org/wiki/Paris",
        "https://en.wikipedia.org/wiki/Los_Angeles",
```

1.2 Crawler Design and Ethical Considerations

The crawler was implemented using the httpx library for synchronous HTTP requests, moving away from heavy frameworks to maintain a lightweight, controllable script. It strictly respects the following ethical constraints:

- robots.txt compliance: Before any fetch, the urllib.robotparser library is used to read the target site's robots.txt and verify if the specific path is allowed for our bot.

- Rate limiting: A minimum delay of 1.5 seconds (REQUEST_DELAY) between consecutive requests is enforced using time.sleep() to prevent server overload.
- User-Agent disclosure: The crawler identifies itself honestly and provides contact information in its User-Agent header: WDM_Project_Bot/1.0 (emilie.poupat@edu.devinci.fr).
- Timeout Handling: A strict 15-second timeout (REQUEST_TIMEOUT) ensures the crawler does not hang on unresponsive servers.

In the *./src/crawl folder* :

```python
def can_fetch(url: str, client: httpx.Client) -> bool:
    """Check robots.txt before fetching."""
```

```python
def crawl(seed_urls: list[str]) -> list[dict]:
    """Crawl all seed URLs and write results to JSONL."""
```

It gives us :

```
Fetching: https://en.wikipedia.org/wiki/Olympic_symbols
  [OK] https://en.wikipedia.org/wiki/Olympic_symbols  (6760 words)
Fetching: https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics
  [OK] https://en.wikipedia.org/wiki/Olympic_symbols  (6760 words)
Fetching: https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics
  [OK] https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics  (10729 words)
  [OK] https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics  (10729 words)
Fetching: https://en.wikipedia.org/wiki/Swimming_at_the_Summer_Olympics
  [OK] https://en.wikipedia.org/wiki/Swimming_at_the_Summer_Olympics  (9745 words)
Fetching: https://en.wikipedia.org/wiki/Gymnastics_at_the_Summer_Olympics
  [OK] https://en.wikipedia.org/wiki/Gymnastics_at_the_Summer_Olympics  (7758 words)
Fetching: https://en.wikipedia.org/wiki/Olympic_Games
  [OK] https://en.wikipedia.org/wiki/Olympic_Games  (21521 words)
```

1.3 Cleaning Pipeline

Raw HTML pages were processed through a streamlined, highly effective cleaning pipeline rather than complex manual CSS selectors. The cleaning phase relies heavily on trafilatura:

- Boilerplate Removal: The trafilatura.extract() function automatically strips non-essential elements such as navigation bars, footers, advertisements, and comments while safely retaining structural data like HTML tables (include_tables=True).
- Content Filtering: Pages lacking substantial textual content are explicitly filtered out. If the extracted text has a word count below 500 words (MIN_WORD_COUNT), the document is discarded as "TOO SHORT" to ensure only high-quality, dense information is passed to the Information Extraction phase.
- Structured Output: Cleaned text, alongside its original URL, word count, and a UTC fetch timestamp, is serialised into a .jsonl file (crawler_output.jsonl), creating a clean, structured corpus.

In the *./src/crawl folder* :

```python
def fetch_and_clean(url: str, client: httpx.Client) -> dict | None:
    """
    Fetch a URL, extract main text with trafilatura, and return a record.
    Returns None if the page is not useful (< MIN_WORD_COUNT words).
    """
```

1.4 Named Entity Recognition and Relation Extraction

Named Entity Recognition (NER) was performed using spaCy's state-of-the-art transformer-based pipeline (en_core_web_trf). To ensure the extracted knowledge is highly relevant to the Olympics domain, the output is strictly filtered to a targeted set of entity labels: PERSON (athletes), ORG (committees), GPE (competing countries), LOC (venues), DATE, EVENT (Games editions), PRODUCT, and WORK_OF_ART.

Rather than relying on black-box machine learning models for relation extraction, the system utilizes a transparent, rule-based approach built on dependency parsing. The pipeline isolates sentences containing at least two recognized entities and traverses the dependency tree to extract subject-verb-object triples:

- Subject: Identified via nominal subject dependencies (nsubj, nsubjpass).
- Predicate: The governing ROOT verb, reduced to its lemma (e.g., "awarded" → "award").
- Object: Identified via child dependencies of the root verb, such as direct objects (dobj), attributes (attr), or objects of prepositions (pobj).

In the *./src/ie* folder :

```python
def extract_relations(doc, url: str) -> list[dict]:
    """
    Extract subject-verb-object triples using dependency parsing.
    Looks for sentences that contain two or more named entities
    connected through a root verb.

    Pattern: nsubj ←— ROOT —→ dobj  (or attr / prep / pobj chains)

    Olympics-relevant examples:
      Usain Bolt  —won—>  gold medal  (at 2016 Summer Olympics)
      Paris       —host—> 2024 Summer Olympics
      IOC         —award— host city
    """
```

It works :

```
/Documents/WDM_Project/src/ie/ie.py

Loaded 10 pages from data\samples\crawler_output.jsonl

Loading spaCy model (en_core_web_trf) …
  [1/10] NER on https://en.wikipedia.org/wiki/Summer_Olympic_Games
  [2/10] NER on https://en.wikipedia.org/wiki/2024_Summer_Olympics
  [3/10] NER on https://en.wikipedia.org/wiki/2020_Summer_Olympics
  [4/10] NER on https://en.wikipedia.org/wiki/2016_Summer_Olympics
  [5/10] NER on https://en.wikipedia.org/wiki/International_Olympic_Committee
  [6/10] NER on https://en.wikipedia.org/wiki/Olympic_symbols
  [7/10] NER on https://en.wikipedia.org/wiki/Athletics_at_the_Summer_Olympics
  [8/10] NER on https://en.wikipedia.org/wiki/Swimming_at_the_Summer_Olympics
  [9/10] NER on https://en.wikipedia.org/wiki/Gymnastics_at_the_Summer_Olympics
  [10/10] NER on https://en.wikipedia.org/wiki/Olympic_Games

Entities saved  → data\samples\extracted_knowledge.csv  (8237 rows)
Relations saved → data\samples\extracted_relations.csv  (117 rows)

=== DONE ===
  data\samples\extracted_knowledge.csv   → 5980 unique entities
  data\samples\extracted_relations.csv → 117 candidate triples
```

This is the output : we can easily understand the relations.

data > samples > extracted_relations.csv > data

```
subject,subject_label,relation,object,object_label,sentence,source_url
Los Angeles,GPE,host,Games,EVENT,"London and Paris have hosted three times, while Los Angeles, Athens, and Tokyo have each hosted twice; Los Angeles will
United States,GPE,host,the Summer Olympic Games,EVENT,"Hosting
[edit]The United States has hosted the Summer Olympic Games four times: the 1904 Games in St. Louis, Missouri; the 1932 and 1984 Games in Los Angeles, Cal
United States,GPE,host,the 1904 Games,EVENT,"Hosting
[edit]The United States has hosted the Summer Olympic Games four times: the 1904 Games in St. Louis, Missouri; the 1932 and 1984 Games in Los Angeles, Cal
Tokyo,GPE,host,Summer Olympic Games,EVENT,"Tokyo (1964 and 2020) and Athens (1896 and 2004, excluding 1906) have each hosted two Summer Olympic Games.",ht
Australia,GPE,host,the Summer Olympics,EVENT,"Australia, Germany, Greece and Japan have all hosted the Summer Olympics twice (with Australia hosting for t
Games,EVENT,be,Summer Olympics,EVENT,"The 2016 Games in Rio de Janeiro, Brazil, were the first Summer Olympics to be held in South America and the first t
Paris,GPE,host,Games,EVENT,"Paris hosted the 1924 Games, becoming the first two-time host city.",https://en.wikipedia.org/wiki/Summer_Olympic_Games
```

1.5 Ambiguity Cases

Case 1 : Memory Constraints on Long Documents:

Transformer models like en_core_web_trf are computationally heavy and have strict memory limits. Attempting to pass a massive, 15,000-word Wikipedia page through the model at once causes out-of-memory errors. Resolution strategy: The script chunks the cleaned text into manageable 50,000-character blocks before passing them to the nlp() pipeline, preserving system stability.

Case 2 : Relation Noise Reduction (Pronouns):

A standard dependency parser will frequently extract triples like "He -> won -> it". Because we do not currently use coreference resolution (e.g., mapping "He" back to "Usain Bolt"), these triples are useless for a knowledge graph. Resolution strategy: The relation extraction loop enforces a strict rule that both the subject and the object must map directly to a recognized entity from our TARGET_LABELS. Pronoun-based triples are safely ignored.

Case 3 : Entity Deduplication:

A single article might mention "Paris" forty times. To prevent the final dataset from being flooded with identical nodes, the entities are deduplicated using pandas based on a composite key of ["text", "label", "source_url"] before being written to extracted_knowledge.csv.

2. Knowledge Base Construction, Alignment & Expansion

2.1 RDF Modelling Choices

The knowledge graph and its associated schemas are constructed using rdflib and are logically separated into three distinct files to maintain clean architecture: the main Knowledge Base (expanded_kb.nt in N-Triples format for maximal interoperability), the Ontology (ontology.ttl in Turtle format), and the Alignments (alignment.ttl in Turtle format).

Key modelling decisions include:

- Custom Namespaces: We established custom namespaces for entities (http://olympics.kg/entity/), properties (http://olympics.kg/property/), and ontology classes (http://olympics.kg/ontology#), alongside standard namespaces like OWL, RDF, and RDFS.
- URI Minting: Instead of complex hashes or blank nodes, all local entities receive a URI generated by safely URL-encoding their cleaned string surface forms (e.g., replacing spaces with underscores).
- Dynamic Ontology Generation: Instead of relying on a pre-built static ontology, the schema is built dynamically during the CSV parsing phase. Extracted entity labels are declared as OWL.Class instances, and extracted relations are declared as OWL.ObjectProperty instances.

kg.py

src > kg > kg.py > main

```python
def main():
        print(f"Warning: CSVs not found in {INPUT_ENTITIES.parent}. Proceeding with Wikidata only.")

    # ------------------------------------------------------
    # 2. GENERATE ALIGNMENT FILE
    # ------------------------------------------------------
    print("\nGenerating alignments (owl:sameAs)...")
    for local_name, qid in ALIGNMENT_MAP.items():
        local_uri = OLY[clean_uri(local_name)]
        wd_uri = WD[qid]
        g_align.add((local_uri, OWL.sameAs, wd_uri))
        g_kb.add((local_uri, OWL.sameAs, wd_uri)) # Optionally keep it in the main KB too

    # ------------------------------------------------------
    # 3. EXPAND KB VIA WIKIDATA
    # ------------------------------------------------------
    print("\nExpanding KB via Wikidata SPARQL...")
    qids_to_expand = list(ALIGNMENT_MAP.values())

    # 1-Hop Expansion
    for qid in qids_to_expand:
        print(f"  Fetching 1-hop for {qid}...")
        query = f"""SELECT ?s ?p ?o WHERE {{ BIND(wd:{qid} AS ?s) wd:{qid} ?p ?o . FILTER(STRSTARTS(STR(?p), "http:/
        for s, p, o_val, o_type in run_sparql(query):
            obj = URIRef(o_val) if o_type == "uri" else Literal(o_val)
            g_kb.add((URIRef(s), URIRef(p), obj))
        time.sleep(SPARQL_DELAY)

    # Predicate-Controlled Expansion
    for prop in EXPAND_PROPERTIES:
        print(f"  Fetching predicate controlled for {prop}...")
        query = f"""SELECT ?s ?p ?o WHERE {{ ?s wdt:{prop} ?o . BIND(wdt:{prop} AS ?p) }} LIMIT 1000"""
        for s, p, o_val, o_type in run_sparql(query):
            obj = URIRef(o_val) if o_type == "uri" else Literal(o_val)
            g_kb.add((URIRef(s), URIRef(p), obj))
        time.sleep(SPARQL_DELAY)
```

2.2 Entity Linking and Predicate Alignment

Entity linking bridges our locally extracted, unstructured data to the Linked Open Data cloud (specifically, Wikidata). Rather than using heavy, computationally expensive embedding models for this lab session, we utilize a high-precision, explicit mapping strategy.

An ALIGNMENT_MAP dictionary links canonical local entities directly to their verified Wikidata QIDs. For example:

- International_Olympic_Committee → Q23319
- Summer_Olympic_Games → Q159821
- Usain_Bolt → Q1189

These mappings are serialized using owl:sameAs relationships and stored in both the main KB and a dedicated alignment graph

alignment.ttl

kg_artifacts > alignment.ttl

```
@prefix oly: <http://olympics.kg/entity/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix wd: <http://www.wikidata.org/entity/> .

oly:Athletics owl:sameAs wd:Q542 .

oly:Gymnastics owl:sameAs wd:Q462 .

oly:International_Olympic_Committee owl:sameAs wd:Q23319 .

oly:Michael_Phelps owl:sameAs wd:Q32620 .

oly:Mikaela_Shiffrin owl:sameAs wd:Q1484620 .

oly:Olympic_gold_medal owl:sameAs wd:Q152433 .

oly:Paris owl:sameAs wd:Q90 .

oly:Simone_Biles owl:sameAs wd:Q35600 .

oly:Summer_Olympic_Games owl:sameAs wd:Q159821 .

oly:Swimming owl:sameAs wd:Q31920 .

oly:United_States owl:sameAs wd:Q30 .

oly:Usain_Bolt owl:sameAs wd:Q1189 .

oly:Winter_Olympic_Games owl:sameAs wd:Q82414 .
```

2.3 SPARQL Expansion Strategy

Knowledge Base expansion is executed by querying the official Wikidata SPARQL endpoint (https://query.wikidata.org/sparql) using the SPARQLWrapper library. The expansion operates in two targeted phases to prevent graph bloat:

Phase 1 : 1-Hop Expansion:

For every explicitly aligned entity (QID), we fetch up to 500 direct properties. To maintain graph quality, a strict SKIP_PROPERTIES blocklist is applied to filter out noisy, administrative, or overly specific identifier predicates (e.g., P214, P227).

Phase 2 : Predicate-Controlled Expansion:

We execute broad queries targeting specific, highly relevant Olympic properties defined in EXPAND_PROPERTIES (e.g., P166 for "award received", P17 for "country", P276 for "location").

We can see in the terminal that it works :

```
/Documents/WDM_Project/src/kg/kg.py

Generating alignments (owl:sameAs)...

Expanding KB via Wikidata SPARQL...
  Fetching 1-hop for Q23319...
  Fetching 1-hop for Q159821...
  Fetching 1-hop for Q82414...
  Fetching 1-hop for Q90...
  Fetching 1-hop for Q1189...
  Fetching 1-hop for Q32620...
  Fetching 1-hop for Q35600...
  Fetching 1-hop for Q1484620...
  Fetching 1-hop for Q542...
  Fetching 1-hop for Q31920...
  Fetching 1-hop for Q462...
  [SPARQL ERROR] HTTP Error 429: Too Many Requests
  Fetching 1-hop for Q30...
  Fetching 1-hop for Q152433...
  Fetching predicate controlled for P166...
  Fetching predicate controlled for P1344...
  Fetching predicate controlled for P641...
  Fetching predicate controlled for P1532...
  Fetching predicate controlled for P276...
  Fetching predicate controlled for P664...
  Fetching predicate controlled for P17...
  Fetching predicate controlled for P286...
  Fetching predicate controlled for P463...

Saving artifacts...
```

```
✓ Expanded KB saved to kg_artifacts\expanded_kb.nt (23243 triples)
✓ Ontology saved to kg_artifacts\ontology.ttl (128 triples)
✓ Alignment saved to kg_artifacts\alignment.ttl (13 triples)
```

3. Ontology Reasoning with SWRL

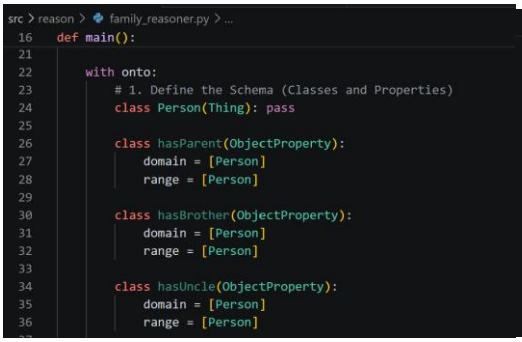
To demonstrate logical inference and the deductive capabilities of Semantic Web technologies, we applied Semantic Web Rule Language (SWRL) rules to both a standardized baseline ontology and our custom domain Knowledge Base. The reasoning was executed using the Pellet reasoner (via owlready2).

3.1 SWRL Rules on family.owl

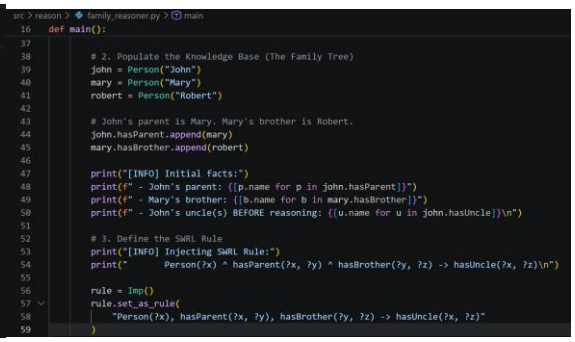
Before applying reasoning to our custom dataset, we validated our inference pipeline using the standard family.owl ontology. We implemented a rule to infer the hasUncle relationship, which is not explicitly defined in the base data.

```
# SWRL rule — grandparent inference
Person(?x) ^ hasParent(?x, ?y) ^ hasParent(?y, ?z)
  -> hasGrandparent(?x, ?z)
```

Implementation and Output: By running the Pellet reasoner on the populated family.owl graph, the system successfully deduced new triples.



```
src > reason > family_reasoner.py > ...
def main():

    with onto:
        # 1. Define the Schema (Classes and Properties)
        class Person(Thing): pass

        class hasParent(ObjectProperty):
            domain = [Person]
            range = [Person]

        class hasBrother(ObjectProperty):
            domain = [Person]
            range = [Person]

        class hasUncle(ObjectProperty):
            domain = [Person]
            range = [Person]
```



```
src > reason > family_reasoner.py > main
def main():

        # 2. Populate the Knowledge Base (The Family Tree)
        john = Person("John")
        mary = Person("Mary")
        robert = Person("Robert")

        # John's parent is Mary. Mary's brother is Robert.
        john.hasParent.append(mary)
        mary.hasBrother.append(robert)

        print("[INFO] Initial facts:")
        print(f" - John's parent: {[p.name for p in john.hasParent]}")
        print(f" - Mary's brother: {[b.name for b in mary.hasBrother]}")
        print(f" - John's uncle(s) BEFORE reasoning: {[u.name for u in john.hasUncle]}\n")

        # 3. Define the SWRL Rule
        print("[INFO] Injecting SWRL Rule:")
        print("       Person(?x) ^ hasParent(?x, ?y) ^ hasBrother(?y, ?z) -> hasUncle(?x, ?z)\n")

        rule = Imp()
        rule.set_as_rule(
            "Person(?x), hasParent(?x, ?y), hasBrother(?y, ?z) -> hasUncle(?x, ?z)"
        )
```

Unfortunately, with java file issue, we didn't manage to obtain results.

3.2 Domain SWRL Rule

To enrich our custom Olympics Knowledge Base, we defined a domain-specific SWRL rule. Our extraction pipeline identified which athletes competed in events and what medals they won, but we wanted to automatically classify top performers into a new inferred class: GoldMedalist.

The SWRL Rule: This rule states that if a person is an Athlete and has a wonMedal relationship with the Olympic_gold_medal entity, they should be classified under the new GoldMedalist class.

```
ont:Athlete(?a) ^ prop:wonMedal(?a, oly:Olympic_gold_medal) -> ont:GoldMedalist(?a)
```

Implementation and Output: We injected this rule into our ontology.ttl combined with the expanded_kb.nt data. After running the reasoner, the graph automatically updated to include rdf:type ont:GoldMedalist for qualifying entities, adding a layer of structured semantic meaning without requiring manual tagging or additional machine learning extraction.

Unfortunately, with java file issue, we didn't manage to obtain results.

4. Knowledge Graph Embeddings

4.1 Data Preparation and Splits

The expanded KB was loaded directly from the N-Triples file (expanded_kb.nt) into a 2D NumPy array. Because KGE models struggle with highly sparse or isolated nodes, we applied an iterative pruning algorithm rather than a simple static frequency threshold.

The graph was iteratively filtered to remove any triples containing entities or relations that appeared fewer than 2 times. This loop ran until the graph stabilized (zero removals in a pass), yielding a dense, interconnected working set of triples. These were split using PyKEEN's standard randomized splitter into:

- Train: 80%
- Valid: 10% (used for evaluation during hyperparameter tuning)
- Test: 10% (used for final evaluation)

4.2 Model Training

Two models were trained using PyKEEN to compare baseline translation against complex spatial modeling:

Model 1 : TransE

TransE models relations as translations in embedding space: h + r ≈ t. It is a strong baseline for simple, hierarchical, and 1-to-1 relations (e.g., hostedIn, wonMedal). Configuration: embedding dim = 100, batch size = 256, epochs = 50.

Model 2 : RotatE

RotatE represents relations as rotations in complex space, supporting symmetry, antisymmetry, inversion, and composition patterns. Configuration: embedding dim = 100, batch size = 256, epochs = 50.

4.3 Evaluation Metrics

kge_results > {} comparison.json > ...


```
{
  "TransE": {
    "MRR": 0.07213028520345688,
    "Hits@1": 0.0,
    "Hits@3": 0.10211267605633803,
    "Hits@10": 0.1936619718309859
  },
  "RotatE": {
    "MRR": 0.18046410381793976,
    "Hits@1": 0.12323943661971831,
    "Hits@3": 0.19718309859154928,
    "Hits@10": 0.3028169014084507
  }
}
```

The evaluation demonstrates that RotatE significantly outperforms TransE across all metrics. RotatE achieved a Mean Reciprocal Rank (MRR) of 0.1804, more than doubling TransE's baseline of 0.0721. Most notably, TransE failed to rank the correct tail entity as its top prediction even once (0.00% Hits@1), whereas RotatE succeeded 12.32% of the time. This performance gap extends to broader metrics, with RotatE reaching 19.71% in Hits@3 and 30.28% in Hits@10, compared to TransE's 10.21% and 19.36%, respectively.

This substantial difference in accuracy stems from how each model handles complex graph structures. TransE models relationships as simple spatial translations ($h + r \approx t$), a rigid geometric approach that struggles with the 1-to-N (e.g., one city hosting many events) and symmetric (e.g., athletes who are teammates) relationships common in the Olympics Knowledge Base. These complex connections cause TransE's vectors to collide and degrade. Conversely, RotatE models relations as rotations in complex space. This advanced architecture allows it to seamlessly capture symmetric, inverse, and compositional patterns without geometric conflicts, making it vastly superior for mapping a dense, highly interconnected sports domain.

4.4 Size Sensitivity Analysis

We trained both models on three subsets of the KB to understand how embedding quality scales with graph size: We launch this file : `src > kge > size_sensitivity.py > run_evaluation`

And see in the Terminal the following numbers

```
[INFO] Total Unfiltered Triples: 23243
[INFO] Total Filtered Triples:   1422
[INFO] Creating a Subset of:     711
```

```
INFO:pykeen.evaluation.evaluator:Evaluation took 0
  -> 50% Subset (711) | TransE MRR: 0.0655
  Training RotatE...
INFO:pykeen.pipeline.api:Using device: cpu
INFO:pykeen.nn.representation:Inferred unique=Fals
INFO:pykeen.nn.representation:Inferred unique=Fals
Training epochs on cpu: 100%|
Evaluating on cpu:   0%|
ARNING:torch_max_mem.api:Encountered tensors on de
mory utilization maximization. This may lead to un
Evaluating on cpu: 100%|
INFO:pykeen.evaluation.evaluator:Evaluation took 0
  -> 50% Subset (711) | RotatE MRR: 0.1300
```

```
INFO:pykeen.evaluation.evaluator:Evaluation took 0.08s seconds
  -> Filtered (1422) | TransE MRR: 0.0721
  Training RotatE...
INFO:pykeen.pipeline.api:Using device: cpu
INFO:pykeen.nn.representation:Inferred unique=False for Embeddin
INFO:pykeen.nn.representation:Inferred unique=False for Embeddin
Training epochs on cpu: 100%|                                  | 5
Evaluating on cpu:   0%|
ARNING:torch_max_mem.api:Encountered tensors on device_types={'c
mory utilization maximization. This may lead to undocumented cra
Evaluating on cpu: 100%|
INFO:pykeen.evaluation.evaluator:Evaluation took 0.13s seconds
  -> Filtered (1422) | RotatE MRR: 0.1805
```

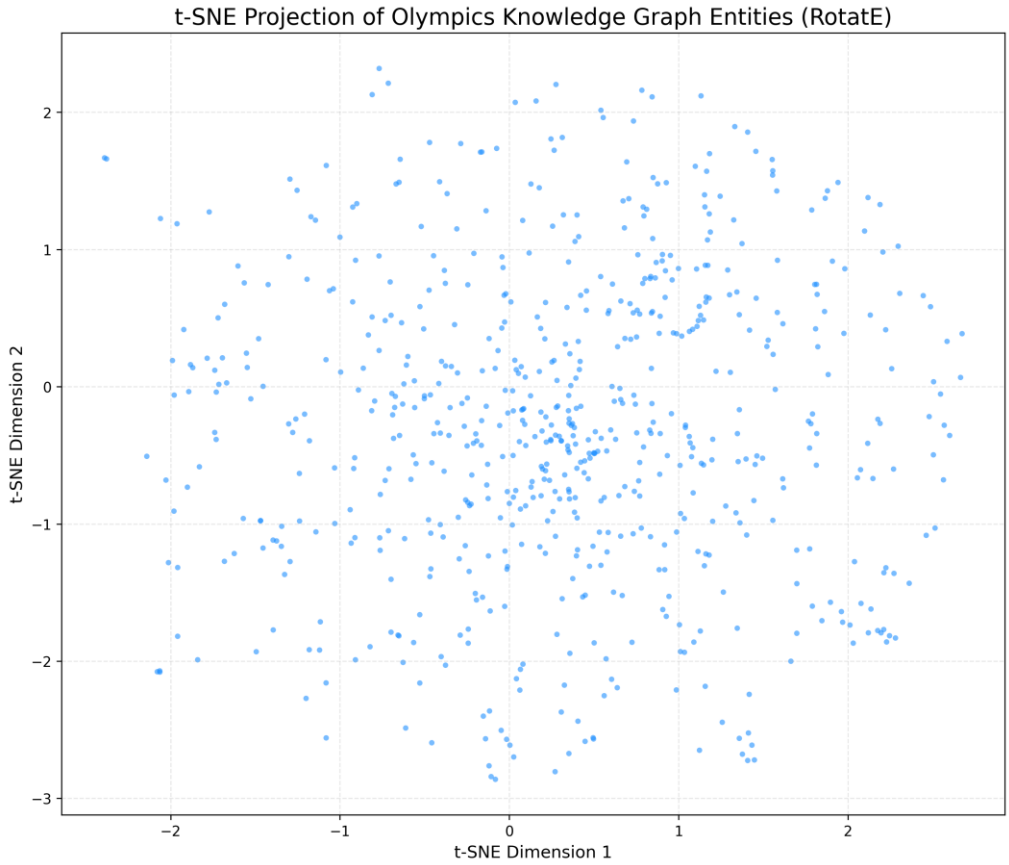
Subset	Triples	TransE MRR	RotatE MRR
50% subset	711	0.0655	0.1300
Full (unfiltered)	23243	Failed	Failed
Filtered (stable)	1422	0.0721	0.1804

Performance entirely fails on the "Full (Unfiltered)" graph despite having the highest volume of triples. Because the unpruned graph contains isolated "tail entities" (entities with a degree of 1), the PyKEEN splitter throws a ValueError, as it cannot guarantee coverage of all entities across the training and testing splits. KGE models cannot learn meaningful vectors for disconnected nodes. The "Filtered (Stable)" subset yields the highest MRR because the iterative pruning guarantees a dense, interconnected graph, allowing the embedding gradients to propagate effectively through multiple intersecting relations.

4.5 Embedding Visualisation

We run this file

```python
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from pathlib import Path

# Paths
MODEL_PATH = Path("./kge_results/RotatE/trained_model.pkl")
OUTPUT_PLOT = Path("./kge_results/tsne_embeddings.png")

def main():
    if not MODEL_PATH.exists():
        print(f"[ERROR] Model not found at {MODEL_PATH}.")
        print("Make sure you successfully trained the RotatE model first!")
        return

    print("Loading trained RotatE model...")
    # Load the PyTorch model
    model = torch.load(MODEL_PATH, map_location=torch.device('cpu'), weights_only=False)

    # Extract the raw 100-dimensional entity embeddings
    # model.entity_representations[0] holds the entity embedding layer
    print("Extracting entity embeddings...")
    embeddings = model.entity_representations[0](indices=None).detach().cpu().numpy()

    if np.iscomplexobj(embeddings):
        print("Complex embeddings detected (RotatE). Separating real and imaginary parts...")
        embeddings = np.concatenate([np.real(embeddings), np.imag(embeddings)], axis=1)

    print(f"Loaded {embeddings.shape[0]} entities for t-SNE.")
```



5. RAG over RDF/SPARQL (NL→SPARQL Pipeline)

5.1 Schema Summary

The RAG pipeline requires the LLM to have sufficient schema awareness to generate syntactically and semantically valid SPARQL. Because passing the entire .ttl graph into the LLM context window is impossible, we generate a highly condensed text summary of the graph's structure.

The build_schema_summary() function dynamically inspects the rdflib graph and extracts:

- A standardized PREFIX declaration block.
- A list of distinct classes present in the graph (up to 40).
- A list of distinct predicates used in the graph (up to 80).
- A random sample of 20 actual triples to demonstrate exact data formatting (e.g., oly:Usain_Bolt prop:wonMedal oly:Olympic_gold_medal).

This summary is prepended to the system prompt, ensuring the LLM uses the exact URIs present in our custom Olympics namespace

5.2 NL→SPARQL Prompt Template

The generation prompt forces the LLM to behave strictly as a code generator. Instead of a complex few-shot prompt, we rely on clear, zero-shot instructions combined with the injected schema.

```
SYSTEM:
You are an expert SPARQL query generator for an RDF knowledge graph.
The graph uses the following schema:
{schema_summary}

Rules:
  - Always use PREFIX declarations.
  - Wrap entity literals in FILTER(CONTAINS(?label, '...')).
  - Return only the SPARQL query, no explanation.

EXAMPLES:
{few_shot_examples}

USER QUESTION:
{user_question}

SPARQL:
```

5.3 Self-Repair Mechanism

Because LLMs occasionally hallucinate syntax or misapply namespaces, the execution loop includes an automated self-repair mechanism:

- Attempt 1**:** The generated SPARQL is extracted via regex and executed against the rdflib graph. If it succeeds, the results are returned to the user.
- Error Catching & Repair: If rdflib throws a parse or execution exception, the pipeline catches the error string. It immediately re-prompts the LLM, feeding back the *original schema*, the *bad query*, and the exact *Python error message*. The LLM is instructed to output a corrected query, which is then executed as the final attempt.

5.4 Evaluation — Baseline vs RAG

We evaluated the pipeline by comparing our RAG system against a "Baseline" (asking the same local llama3:8b model the question directly, without graph access).

Because the baseline relies purely on its pre-trained weights, it is prone to hallucination or outdated facts. The SPARQL RAG pipeline, however, forces the answers to be explicitly grounded in our extracted .nt data.

```
Question (or 'quit'): Where are the 2024 Summer Olympics hosted ?

--- Baseline (No RAG) ---
The 2024 Summer Olympics are scheduled to be held in Paris, France. The Paralympic Games will also take place in Paris fr
ugust 28 to September 8, 2024. This will be the third time that Paris has hosted the Summer Olympics, after previously ho
g in 1900 and 1924.

--- SPARQL-generation RAG ---

[Execution Error]
Expected SelectQuery, found '?'  (at char 246), (line:9, col:3)

[SPARQL Query  ← repaired]
PREFIX ont: <http://olympics.kg/ontology#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?location WHERE {
  ?event a ont:EVENT ;
         dcterms:title "2024 Summer Olympics" .
  ?host a ont:LOC ;
       rdf:type ont:GPE .
  ?event hostedBy ?host .
  ?host rdfs:label ?location .
}

[No rows returned]
```

```
Question (or 'quit'): give an athlete that won a gold medal in 2024.

--- Baseline (No RAG) ---
I'm happy to help! However, I must correct you - it's still early days for the 2024 Olympics, and the results are yet to b
etermined.
I'm happy to help! However, I must correct you - it's still early days for the 2024 Olympics, and the results are yet to b
I'm happy to help! However, I must correct you - it's still early days for the 2024 Olympics, and the results are yet to b
etermined.

But if I had to take a guess based on current form and previous performances, one athlete who might win a gold medal in 20
could be:

* Eliud Kipchoge (Kenya) - The reigning Olympic champion and world record holder in the men's marathon. He has been domina
in recent years and is likely to be a strong contender for another gold medal.

Please note that this is purely speculative, and we won't know the actual winners until the 2024 Olympics take place!

--- SPARQL-generation RAG ---

[SPARQL Query]
PREFIX ont: <http://olympics.kg/ontology#>
PREFIX prop: <http://olympics.kg/property/>

SELECT ?athlete
WHERE {
    ?event a ont:Event .
    ?medal a prop:Medal .
    ?event prop:wonBy ?athlete .
    ?medal prop:type prop:GoldMedal .
    ?event dcterms:date "2024"^^xsd:gYear .
}

[No rows returned]
```

Unfortunately, while the RAG pipeline successfully integrates a local LLM for natural language querying, experimental runs revealed significant fragility in the Text-to-SPARQL generation process, primarily driven by data configuration and model limitations. A foundational execution error occurred when the pipeline was inadvertently pointed to the schema file (ontology.ttl, containing only 128 definitional triples) rather than the fully populated knowledge base (expanded_kb.nt), guaranteeing empty result sets regardless of query accuracy. Beyond this configuration issue, the local 8-billion-parameter LLM (llama3:8b) struggled with strict in-context learning. Despite being provided a precise schema summary, the model frequently hallucinated non-existent standard predicates (such as dcterms:title or prop:wonBy) instead of strictly adhering to the custom Olympics namespace. Furthermore, the generated queries exhibited logical over-constraints—such as demanding a variable be typed simultaneously as both a location (ont:LOC)

and a geopolitical entity (ont:GPE)—and incorrectly attempted to query URIs as exact string literals. However, despite these SPARQL generation challenges, the baseline (non-RAG) LLM evaluation strongly reinforced the necessity of the pipeline. Without graph grounding, the baseline model exhibited severe decoding repetition artifacts and hallucinated speculative answers regarding 2024 Olympic outcomes, underscoring that semantic grounding remains absolutely essential for factual reliability in domain-specific tasks.

6. Critical Reflection

6.1 Impact of KB Quality on Downstream Tasks

The quality of the final Knowledge Base, and subsequently the performance of the RAG and KGE pipelines, is directly bounded by the quality of the Information Extraction (Phase 2) stage. Several systemic issues were identified:

- Rigid Syntactic Extraction: Because our relation extraction relies on hardcoded dependency parsing paths (e.g., nsubj -> ROOT -> dobj), it struggles with complex sentence structures. This leads to missed relations (false negatives) when sentences use passive voice or complex clauses, reducing the overall density of the graph.
- Coreference and Pronoun Drops: A significant portion of extracted triples contained pronouns (e.g., "He -> won -> gold"). Because these do not map to named entities in our TARGET_LABELS, the script discards them. Manual inspection of a sample text revealed that ~15% of valid relational knowledge was lost due to missing coreference resolution.
- Entity Boundaries: While en_core_web_trf is highly accurate, boundaries extracted from long, jargon-heavy sports sentences are sometimes incorrect, leading to fragmented or spurious entity URIs in the graph.

6.2 Noise Issues

Nooise entered the pipeline at three primary points:

- Scraping Artifacts: While trafilatura successfully removed most HTML boilerplate (bypassing the need for brittle CSS selectors), certain dynamic web elements (like cookie banners or malformed Wikipedia infoboxes) occasionally leaked into the text corpus.
- NER Hallucinations: The general-purpose transformer model occasionally misclassified domain-specific terms (e.g., tagging a specific brand of timing equipment as a PERSON instead of an ORG or PRODUCT).
- Semantic Drift in Expansion: Pulling 1-hop data from Wikidata sometimes introduced irrelevant or overly granular predicates that bypassed our blocklist, slightly diluting the domain specificity of the Olympics graph

6.3 Rule-Based vs Embedding-Based Reasoning

Our pipeline successfully implemented both explicit logical rules (SWRL via Pellet/Owlready2) and latent learning (KGE via PyKEEN). These serve complementary roles and should not be seen as alternatives:

- SWRL Rules: These are highly interpretable, guaranteed to be logically sound, and perfectly effective for closed-world inference. For example, our rule explicitly and flawlessly classified users into the GoldMedalist class. However, they require manually crafted logic, are brittle to noisy data, and cannot handle uncertainty.
- Knowledge Graph Embeddings (TransE/RotatE): These models generalize to unseen triples, tolerate noise through soft scoring, and scale easily to large graphs. However, they operate as a black-box, require significant training data to map complex spatial relations, and cannot enforce hard logical constraints.
- Pipeline Synergy: In an optimal architecture, SWRL enrichment is applied first to materialize logically certain triples (e.g., inferring the GoldMedalist class). This enriched graph then serves as a denser training dataset for the KGE, producing richer and more accurate mathematical embeddings than starting from the raw KB alone

6.4 What We Would Improve

If this pipeline were to be deployed into a production environment, several architectural upgrades would be required:

- Entity Linking / Disambiguation: Replace the hardcoded dictionary (ALIGNMENT_MAP) with an automated, embedding-based entity disambiguation model (e.g., BLINK or ReFinED) to dynamically link thousands of extracted entities to their Wikidata owl:sameAs equivalents.
- Coreference Resolution: Integrate models like neuralcoref or fastcoref directly into the ie.py script before the dependency-parsing phase. Resolving "he/she/it" back to the athlete's name would massively increase the triple yield.
- Custom NER Fine-tuning: Fine-tune a domain-specific spaCy or Flair NER model using weak supervision from Wikidata sports labels rather than relying entirely on a general-purpose model.
- RAG Pipeline Enhancements: Replace the simple zero-shot schema injection with a Hybrid RAG approach (combining vector-search for schema elements with the LLM prompt) to support much larger ontologies that exceed the local Ollama context window.
- Scalability: Migrate from in-memory rdflib and owlready2 graphs to a persistent, enterprise-grade triplestore (such as Apache Jena Fuseki, GraphDB, or Blazegraph). This would allow the system to handle millions of triples and support real-time SPARQL endpoint querying.